COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	10/ 14 / 26	**DATE SUBMITTED:**	10/ 16 / 26
NAME:	Bandigan , Christian Nieles, Cenndy	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 12 **Hands-on Activity 12.1 Algorithmic Strategies**

1. PROCEDURE OUTPUT

ILO A

Strategy	Algorithm	Analysis
Recursion	Quick Sort	In a previous activity a Quick Sort applied this recursive strategy by selecting a pivot element partitioning the array into sub-arrays of elements smaller and greater than the pivot .
Brute Force	Bubble Sort	Bubble Sort it's a straightforward sorting algorithm that systematically iterates through the list compared to adjacent elements, and swaps . It employs a brute force approach by exhaustively checking and re-checking.
Backtracking	Post-Order Traversal	Post-Order is a traversal algorithm for trees that incorporates backtracking. It follows a left-right-root policy meaning it visits the child nodes first and backtracks back to the root node.
Greedy	Traversing Weighted Trees	A greedy strategy is used when traversing weighted trees by repeatedly choosing the node with the smallest cost.
Divide-and-Conquer	Merge Sort	In a previous activity, we were able to implement a merge sort which uses a divide-and-conquer approach. To sort elements, merge sort recursively calls to divide the array in halves until one

		remains and is considered sorted. After that it merges all the array until you have a sorted array

Table 12-1. Algorithmic Strategies and Examples

Screenshot	HOA12 > memoization.cpp > main()
	1 #include <iostream>
	2 #include <algorithm>
	3
	4 const int size = 100;
	5 int memArr[size];
	6
	7 int getMinSteps(int n);
	8
	9 int main(){
	10 // initialize memoized array
	11 for (int i=0; i<=size; i++)
	12 memArr[i] = -1;
	13
	14 std::cout << getMinSteps(3) << std::endl;
	15 std::cout << getMinSteps(11) << std::endl;
	16
	17 return 0;
	18 }
	19
	20 int getMinSteps (int n){
	21 if (n == 1) return 0; // base case
	22 if(memArr[n] != -1){
	23 return memArr[n]; // we have solved it already :)
	24 }
	25 int r = 1 + getMinSteps(n - 1); // '-1' step . 'r' will contain the optimal answer finally
	26 if(n%2 == 0) r = std::min(r , 1 + getMinSteps(n / 2)); // '/2' step
	27 if(n%3 == 0) r = std::min(r , 1 + getMinSteps(n / 3)) ; // '/3' step
	28 memArr[n] = r ; // save the result. If you forget this step, then its same as plain recursion.
	29 return r;
	30 }
	Output
	PS C:\Users\User\OneDrive\Documents\CPE010_DSA> cd "c:\
	1
	4

Analysis	This algorithm uses a dynamic programming algorithm specifically it implements memoization. After solving a problem it stores the answer in an array so it can be used in the future. Best-Case Time Complexity: O(1) Worst-Case Time Complexity: O(n)

Table 12-2. Memoization Implementation

Screenshot	HOA12 > main.cpp > getMinsSteps(int) *Output* PS C:\Users\TIPQC\Documents\CPE21S2-NIELES\HOA12> ./program.exe 3
Analysis	*This algorithm it implements a bottom-up dynamic programming approach to find the minimum steps required to reduce a given number n to 1*

```cpp
#include <iostream>
#include <algorithm>

int getMinsSteps(int n) {

    int*  dp = new int[n + 1];

    dp[1] = 0;

    for (int i = 2; i <= n; i++) {
        dp[i] = 1 + dp[i - 1];
        if (i % 2 == 0) dp[i] = std::min(dp[i], 1 + dp[i / 2]);
        if (i % 3 == 0) dp[i] = std::min(dp[i], 1 + dp[i / 3]);
    }

    int result = dp[n];

    delete[] dp;

    return result;
}

int main() {
    std::cout << getMinsSteps(10) << std::endl;
    return 0;
}
```

Table 12-3. Bottom-Up Dynamic Programming Implementation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Pseudocode	START INITIALIZE 4x4 matrix FUNCTION pathCount(cost,row index, column index) IF index is out of bounds RETURN 0 SUBTRACT cost to the current cost IF cost < 0 RETURN 0 IF in last cell IF cost == 0 RETURN 1 ELSE RETURN 0

	SET right to pathCount(cost, row index,column index +1) SET left to pathCount(cost, row index+1, column index) RETURN right + left END FUNCTION END
Code	HOA12 > supplementary > iloc.cpp > ...

```cpp
#include <iostream>

const int col = 4;
const int row = 4;

int matrix[row][col] = {
        {4,7,1,6},
        {6,7,3,9},
        {3,8,1,2},
        {7,1,7,3}
    };

int pathCount(int cost, int i, int j);

int main(){
    int cost = 25;
    std::cout << "Cost: " << cost <<  std::endl;
    std::cout<< "Paths: " << pathCount(cost,0,0);

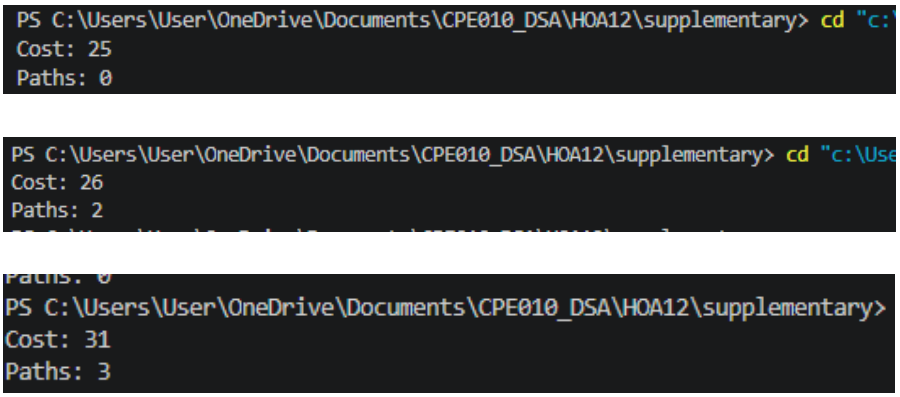
    return 0;
}

int pathCount(int cost, int i, int j){
    if(i >= row || j >= col){
        return 0;
    }
    cost -= matrix[i][j];

    if(cost < 0){
        return 0;
    }

    if(i == row-1 && j == col-1){
        if(cost == 0){
            return 1;
        }else{
            return 0;
        }
    }
    int right = (pathCount(cost,i,j+1));
    int down = (pathCount(cost,i+1,j));

    return right + down;
}
```

Demonstration	
Analysis	**Bandigan** - The algorithm uses recursion and backtracking to explore all possible paths that are based on the cost and count them. Each time the function visits a cell it subtracts the cell value from the remaining costs. It has 3 base cases to end the function for the index going out of bounds, the cost becomes negative, and when it reaches the last cell. Best-Case Time Complexity: O(M + N) Worst-Case Time Complexity: $O(2^{M+N})$ **Nieles** -

Conclusion

Bandigan

- In this activity we learned about different algorithmic strategies such as recursion, backtracking, divide-and-conquer, dynamic programming, brute force, and greedy. We were able to look back at our past activities and find algorithms that use these strategies and analyze them. We were also able to create a program that implements dynamic programming such as memoization and bottom-up approach which basically just remembers past results for future use. Moreover, in the supplementary activity, we were able to solve a problem using what we learned about algorithmic strategies.

Nieles

-

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

COMPUTER ENGINEERING

QUEZON CITY

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.